

Лабораторная работа

**Исследование работы механизмов
передачи сообщений и сигналов в ОС
QNX**

Методические указания

1 Цель работы

Познакомиться с механизмами обмена сообщениями в ОС QNX.

2 Связь между процессами

2.1 Связь между процессами посредством сообщений

Механизм передачи межпроцессных сообщений занимается пересылкой сообщений между процессами и является одной из важнейших частей операционной системы, так как все общение между процессами, в том числе и системными, происходит через сообщения. Сообщение в *QNX* – это последовательность байтов произвольной длины (0-65535 байтов) и произвольного формата.

Протокол обмена сообщениями выглядит таким образом: задача блокируется для ожидания сообщения. Другая же задача посылает первой сообщение и при этом блокируется сама, ожидая ответа. Первая задача становится разблокированной, обрабатывает сообщение и отвечает, разблокируя при этом вторую задачу.

Сообщения и ответы, пересылаемые между процессами при их взаимодействии, находятся в теле отправляющего их процесса до того момента, когда они могут быть приняты. Это значит, что, с одной стороны, уменьшается вероятность повреждения сообщения в процессе передачи, а с другой – уменьшается объем оперативной памяти, необходимый для работы ядра. Кроме того, уменьшается число пересылок из памяти в память, что разгружает процессор. Особенностью процесса передачи сообщений является то, что в сети, состоящей из нескольких компьютеров под управлением *QNX*, сообщения могут прозрачно передаваться процессам, выполняющимся на любом из узлов.

Передача сообщений служит не только для обмена данными между процессами, но, кроме того, является средством синхронизации выполнения нескольких взаимодействующих процессов.

Рассмотрим более подробно функции *Send*, *Receive* и *Reply*.

Использование функции *Send*. Предположим, что процесс *A* выдает запрос на передачу сообщения процессу *B*. Запрос оформляется вызовом функции *Send*.

Функция *Send* имеет следующие аргументы:

- *pid* — идентификатор процесса-получателя сообщения (т.е. процесса *B*);

pid — это идентификатор, посредством которого процесс опознается операционной системой и другими процессами;

- *smsg* — буфер сообщения (т.е. посылаемого сообщения);
- *rmsg* — буфер ответа (т.е. сообщения посылаемого в ответ);
- *smsg_len* — длина посылаемого сообщения;
- *rmsg_len* — максимальная длина ответа, который должен получить процесс

A.

Обратите внимание на то, что в сообщении будет передано не более чем *smsg_len* байт и принято в ответе не более чем *rmsg_len* байт — это служит гарантией того, что буферы никогда не будут переполнены.

Использование функции *Receive*. Процесс *B* может принять запрос *Send*, выданный процессом *A*, с помощью функции *Receive*.

Функция *Receive* имеет следующие аргументы:

- *pid* — идентификатор процесса, пославшего сообщение (т.е. процесса *A*);
- *0* — (ноль) указывает на то, что процесс *B* готов принять сообщение от любого процесса;
- *msg* — буфер, в который будет принято сообщение;
- *msg_len* — максимальное количество байт данных, которое может поместиться в приемном буфере.

В том случае, если значения *smsg_len* в функции *Send* и *msg_len* в функции *Receive* различаются, то количество передаваемых данных будет определяться наименьшим из них.

Использование функции *Reply*. После успешного приема сообщения от процесса *A* процесс *B* должен ответить ему, используя функцию *Reply*.

Функция *Reply* имеет следующие аргументы:

- *pid* — идентификатор процесса, которому направляется ответ (т.е. процесса *A*);
- *reply* — буфер ответа;
- *reply_len* — длина сообщения, передаваемого в ответе.

Если значения *reply_len* в функции *Reply* и *rmsg_len* в функции *Send* различаются, то количество передаваемых данных определяется наименьшим из них.

Дополнительные возможности передачи сообщений. В системе *QNX* имеются функции, предоставляющие дополнительные возможности передачи сообщений, а именно:

- условный прием сообщений;
- чтение и запись части сообщения;
- передача составных сообщений.

Обычно для приема сообщения используется функция *Receive*. Этот способ приема сообщений в большинстве случаев является наиболее предпочтительным.

Однако иногда процессу требуется предварительно «знать», было ли ему послано сообщение, чтобы не ожидать поступления сообщения в *RECEIVE*-блокированном состоянии. Например, процессу требуется обслуживать несколько высокоскоростных устройств, не способных генерировать прерывания и, кроме того, процесс должен отвечать на сообщения, поступающие от других процессов. В этом случае используется функция *Creceive* (Попробовать послать сообщение), которая считывает сообщение, если оно становится доступным, или немедленно возвращает управление процессу, если нет ни одного отправленного сообщения.

Примечание. По возможности следует избегать использования функции *Creceive*, так как она позволяет процессу непрерывно загружать процессор на соответствующем приоритетном уровне.

2.2 Связь между процессами посредством сигналов

Связь посредством сигналов представляет собой традиционную форму асинхронного взаимодействия, используемую в различных операционных системах.

В системе *QNX* поддерживается большой набор *POSIX*-совместимых сигналов, специальные *QNX*-сигналы, а так же исторически сложившиеся сигналы, используемые в некоторых версиях системы *UNIX*.

Сигнал выдается процессу при наступлении некоторого заранее определенного для данного сигнала события. Процесс может выдать сигнал самому себе.

Если вы хотите сгенерировать сигнал из интерпретатора *Shell*, используйте утилиты *kill()* или *slay()*.

В зависимости от того, каким образом был определен способ обработки сигнала, возможны три варианта его приема:

- Если процессу не предписано выполнять каких-либо специальных действий по обработке сигнала, то по умолчанию поступление сигнала прекращает выполнение процесса;
- Процесс может проигнорировать сигнал. В этом случае выдача сигнала не влияет на работу процесса (обратите внимание на то, что сигналы *SIGCONT*, *SIGKILL* и *SIGSTOP* не могут быть проигнорированы при обычных условиях);
- Процесс может иметь обработчик сигнала, которому передается управление при поступлении сигнала. В этом случае говорят, что процесс может

«ловить» сигнал. Фактически такой процесс выполняет обработку программного прерывания. Данные с сигналом не передаются.

Интервал времени между генерацией и выдачей сигнала называется задержкой. В данный момент времени для одного процесса могут быть задержаны несколько разных сигналов. Сигналы выдаются процессу тогда, когда планировщик ядра переводит процесс в состояние готовности к выполнению. Порядок поступления задержанных сигналов не определен.

Иногда может потребоваться временно задержать выдачу сигнала, не изменяя при этом способа его обработки. В системе *QNX* имеется набор функций, которые позволяют блокировать выдачу сигналов. После разблокировки сигнал выдается программе.

Во время работы обработчика сигналов *QNX* автоматически блокирует обрабатываемый сигнал. Это означает, что не требуется организовывать вложенные вызовы обработчика сигналов. Каждый вызов обработчика сигналов не прерывается остальными сигналами данного типа. При нормальном возврате управления от обработчика, сигнал автоматически разблокируется.

Существует важная взаимосвязь между сигналами и сообщениями. Если при генерации сигнала ваш процесс окажется *SEND*-блокированным или *RECEIVE*-блокированным (причем имеется обработчик сигналов), то будут выполняться следующие действия:

- процесс разблокируется;
- выполняется обработка сигнала;
- функции *Send* или *Receive* возвращают управление с кодом ошибки.

Если процесс был *SEND*-блокированным, то проблемы не возникает, так как получатель не получит сообщение. Но если процесс был *REPLY*-блокированным, то неизвестно, было, обработано отправленное сообщение или нет, а, следовательно, неизвестно, нужно ли еще раз выдавать *Send*.

Процесс, выполняющий функции сервера (т.е. принимающий сообщения), может запрашивать уведомления о том, когда обслуживаемый процесс выдаст сигнал, находясь в *REPLY*-блокированном состоянии. В этом случае обслуживаемый процесс становится *SIGNAL*-блокированным с задержанным сигналом, и обслуживающий процесс принимает специальное сообщение, описывающее тип сигнала. Обслуживающий процесс может выбрать одно из следующих действий:

- нормально завершить первоначальный запрос: отправитель будет уведомлен о том, что сообщение было обработано надлежащим образом;

- освободить все закрепленные ресурсы и вернуть управление с кодом ошибки, указывающим на то, что процесс был разблокирован сигналом: отправитель получит чистый код ошибки.

Когда обслуживающий процесс сообщает другому процессу, что он *SIGNAL*-блокирован, сигнал выдается немедленно после возврата управления функцией *Send*.

3 Примеры обмена сообщениями

Минимальный полезный набор функций в *QNX Neutrino* включает в себя функции *ChannelCreate()*, *ConnectAttach()*, *MsgReply()*, *MsgSend()* и *MsgRecieve()*.

Клиент

Клиент, который желает послать запрос серверу, блокируется до тех пор, пока сервер не завершит обработку запроса. Затем, после завершения сервером обработки, запроса клиент разблокируется, чтобы принять ответ.

Это подразумевает обеспечение двух условий: клиент должен сначала установить соединение с сервером, а потом обмениваться с ним данными с помощью сообщений — как в одну сторону (запрос — *send*), так и в другую (ответ — *reply*).

Установление соединения

Для установки соединения между клиентом и сервером используется функция *ConnectAttach()*, описанная следующим образом:

```
#include <sys/neutrino.h>

int ConnectAttach (int nd, pid_t pid, int chid, unsigned
index, int flags);
```

Функции *ConnectAttach()* передаются три идентификатора;

nd — дескриптор узла (*Node Descriptor*),

pid — идентификатор процесса (*Process ID*)

chid — идентификатор канала (*Channel ID*).

Вместе эти три идентификатора, которые обычно записываются в виде «*ND/PID/CHID*», однозначно идентифицируют сервер, с которым клиент желает соединиться.

Выполнив все, для чего предназначалось соединение, его можно уничтожить с помощью функции:

```
ConnectDetach (coid);
```

Передача сообщений (sending)

Передача сообщения со стороны клиента осуществляется с помощью функции `MsgSend()`:

```
#include <sys/neutrino.h>

int MsgSend (int coid, const void *smsg, int sbytes, void
*rmsg, int rbytes);
```

Аргументами функции `MsgSend()` являются:

`coid` — идентификатор соединения с целевым сервером;

`smsg` — указатель на передаваемое сообщение;

`sbytes` — размер передаваемого сообщения;

`rmsg` — указатель на буфер для ответного сообщения;

`rbytes` — размер ответного сообщения.

После приема сообщения сервер обрабатывает его и в некоторый момент времени выдает ответ с результатами обработки. В этот момент функция `MsgSend()` должна вернуть ноль (0), указывая этим, что все прошло успешно.

Если бы сервер послал в ответ какие-то данные, можно было бы вывести их на экран.

Сервер. Создание канала

Сервер должен создать канал — то, к чему бы присоединялся клиент, когда вызывал функцию `ConnectAttach()`.

Канал создается с помощью функции `ChannelCreate()` и уничтожается с помощью функции `ChannelDestroy()`:

```
#include <sys/neutrino.h>

int ChannelCreate (unsigned flags);
int ChannelDestroy (int chid);
```

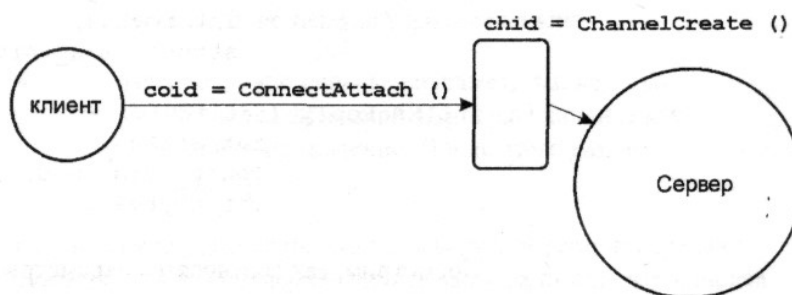


Рис. 1. Связь между каналом сервера и клиентским соединением

Обработка сообщений

В терминах обмена сообщениями, сервер отрабатывает схему обмена в два этапа:

этап приема (*receive*);

этап ответа (*reply*),
используя для этого два простейших варианта соответствующих функций,
MsgReceive() и **MsgReply()**.

```
#include <sys/neutrino.h>

int MsgReceive (int chid, void *rmsg, int rbytes, struct
_msg_info *info);

int MsgReply (int rcvid, int status, const void *msg, int
nbytes);
```

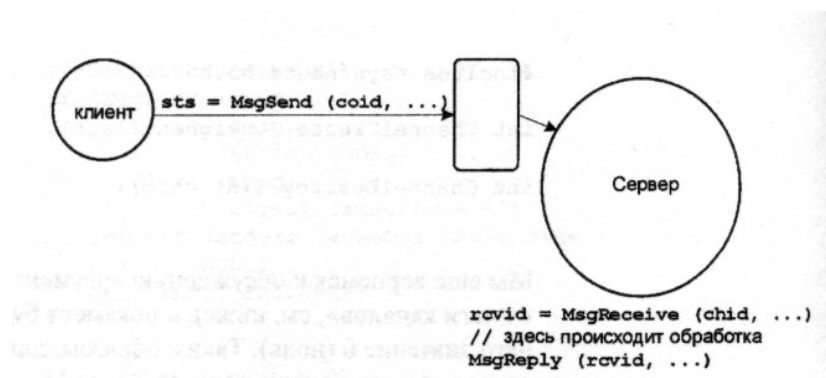


Рис. 2. Взаимосвязь функций клиента и сервера при обмене сообщениями

Для каждой буферной передачи указываются два размера (в случае запроса от клиента это **sbytes** на стороне клиента и **rbytes** на стороне сервера; в случае ответа сервера это **sbytes** на стороне сервера и **rbytes** на стороне клиента). Это сделано для того, чтобы разработчики каждого компонента смогли определить размеры своих буферов — из соображений дополнительной безопасности.

Клиент вызывает функцию **MsgSend()** и указывает ей на буфер передачи (указателем **smsg** и длиной **sbytes**). Данные передаются в буфер функции **MsgReceive()** на стороне сервера, по адресу **rmsg** и длиной **rbytes**. Клиент блокируется.

Функция **MsgReceive()** сервера разблокируется и возвращает идентификатор отправителя **rcvid**, который будет впоследствии использован для ответа. Теперь сервер может использовать полученные от клиента данные.

Сервер завершил обработку сообщения и теперь использует идентификатор отправителя **rcvid**, полученный от функции **MsgReceive()**, передавая его функции **MsgReply()**. Местоположение данных для передачи функции **MsgReply()** задается как указатель на буфер (**smsg**) определенного размера (**sbytes**). Ядро передает данные клиенту.

Ядро передает параметр **sts**, который используется функцией **MsgSend()** клиента как возвращаемое значение. После этого клиент разблокируется.

3.3 Определение идентификаторов узла, процесса и канала (ND/PID/CHID) нужного сервера

Для соединения с сервером функции `ConnectAttach()` необходимо указать дескриптор узла (*Node Descriptor* — *ND*), идентификатор процесса (*Process ID* — *PID*), а также идентификатор канала (*CHannel ID* — *CHID*).

Если один процесс создает другой процесс, тогда это просто — вызов создания процесса возвращает идентификатор вновь созданного процесса. Создающий процесс может либо передать собственные *PID* и *CHID* вновь созданному процессу в командной строке, либо вновь созданный процесс может вызвать функцию `getppid()` для получения идентификатора родительского процесса, и использовать некоторый известный идентификатор канала.

```
#include <process.h>
pid_t getppid( void );
```

Способы объявления сервером *ND/PID/CHID*:

1. Открыть файл с известным именем и сохранить в нем *ND/PID/CHID*. Такой метод является традиционным для серверов *UNIX*, когда сервер открывает файл (например, `/etc/httpd.pid`), записывает туда свой идентификатор процесса в виде строки *ASCII* и предполагают, что клиенты откроют этот файл, прочитают из него идентификатор.

2. Использовать для объявления идентификаторов *ND/PID/CHID* глобальные переменные. Такой способ обычно применяется в многопоточных серверах, которые могут посылать сообщение сами себе. Этот вариант по самой своей природе является очень редким.

3. Занять часть пространства имен путей и стать администратором ресурсов.

Пример 1.1 (server.c):

```
#include <sys/neutrino.h>
#include <sys/iomsg.h>
#include <errno.h>
#include <stdio.h>
#include <string.h>
void main(void)
{
    int rcvid; // идентификатор отправителя
    int chid; // идентификатор канала
    char mess [512];
    printf("Это сервер \n");
    chid = ChannelCreate(0); // Создать канал
    printf("Идентификатор канала: %d \n", chid);
    printf("Идентификатор процесса-сервера: %d\n", getpid());
    while(1)
    {
        rcvid = MsgReceive(chid, mess, sizeof(mess), NULL);
        printf("Получил сообщение от клиента %d\n", rcvid);
        printf("Сообщение такое: \"%s\", \n", mess);
        strcpy(mess, "Это ответ");
        MsgReply(rcvid, EOK, mess, sizeof(mess));
        printf("Отправил сообщение клиенту %d\n", rcvid);
        printf("Сообщение такое: \"%s\" \n", mess);
    }
}
```

Как видно из программы, функция **MsgReceive()** сообщает ядру том, что она может обрабатывать сообщения размером вплоть до **sizeof(message)** (или 512 байт).

Пример 1.2 (client.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/neutrino.h>
#include <string.h>

int main(void) {
    char smsg[20];
    char rmsg[200];
    int coid;
    long serv_pid;
    int chid;
    printf("Это клиент. Введите PID процесса-сервера: ");
    scanf("%ld", &serv_pid);
    printf("Введите идентификатор канала CHID: \n");
    scanf("%d", &chid);
    coid = ConnectAttach(0, serv_pid, chid, 0, 0);
    if (coid == -1) {
        printf("Ошибка %d соединения с сервером %d по каналу %d\n",
            stderr, serv_pid, chid);
        exit(EXIT_FAILURE);
    }
    printf("Идентификатор соединения coid %d \n", coid);
    printf("Введите сообщение: ");
    scanf("%s", &smsg);
    if (MsgSend(coid, smsg, strlen(smsg)+1, rmsg, sizeof(rmsg)) == -1) {
        printf("Ошибка послыки сообщения: %d \n", stderr);
        exit (EXIT_FAILURE);
    }
    if (strlen (rmsg) > 0) printf ("Сообщение %s от сервера с PID %d \n",
        serv_pid, rmsg);
    return(1);
}
```

После компиляции программ сервера и клиента у нас будет 2 исполняемых файла. Назовём их *server* и *client* соответственно.

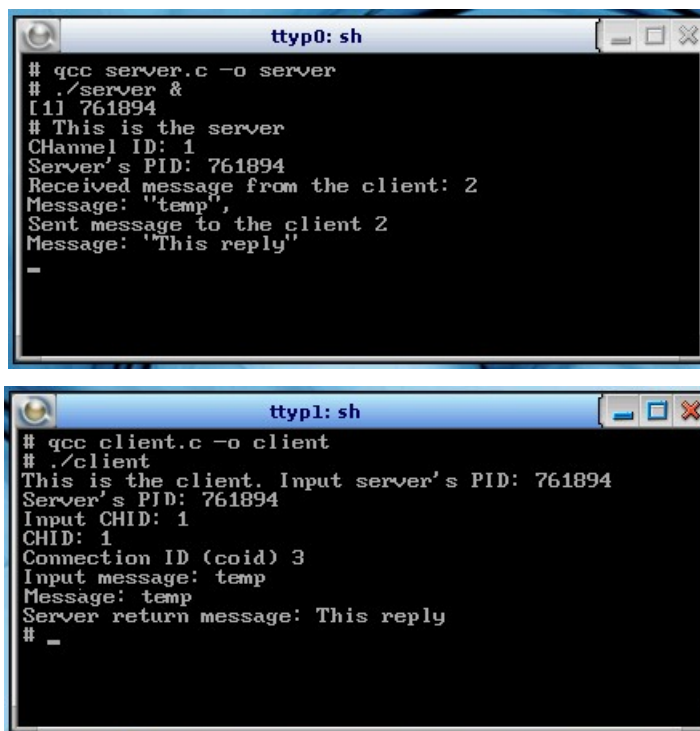
Программу *server* необходимо запустить в фоновом режиме **#server&**. Она начнёт работать: создаст канал, выведет номер канала и идентификатор процесса сервера, будет ждать сообщения от клиента.

Потом необходимо запустить программу *client*. Она создаст процесс клиент и попросит ввести идентификатор процесса сервера, для установления

соединения с ним, и само сообщение (20 символов). Далее можно наблюдать, как сервер получит сообщение, выведет его и пошлёт ответ.

На этом клиент закончит свою работу, но его можно запустить ещё раз и послать другое сообщение.

Предположим, что процесс с идентификатором **77** был действительно активным сервером, ожидающим сообщение именно такого формата по каналу с идентификатором **1**.



```
ttyp0: sh
# gcc server.c -o server
# ./server &
[1] 761894
# This is the server
Channel ID: 1
Server's PID: 761894
Received message from the client: 2
Message: "temp",
Sent message to the client 2
Message: "This reply"
_

ttyp1: sh
# gcc client.c -o client
# ./client
This is the client. Input server's PID: 761894
Server's PID: 761894
Input CHID: 1
CHID: 1
Connection ID (coid) 3
Input message: temp
Message: temp
Server return message: This reply
# _
```

Остановить работу сервера можно функцией **kill <идентификатор процесса сервера>**.

4 Примеры обмена сигналами

POSIX допускает, что не все сигналы могут быть реализованы. Более того, допускается ситуация, когда некоторое символическое имя сигнала определено, но сам сигнал отсутствует в системе (например, при переходе от одной версии **QNX** к другой). Для диагностики реального наличия сигнала можно воспользоваться рекомендацией, приведенной в стандарте **POSIX** 1003.1: наличие поддержки сигнала сообщает вызов функции **sigaction()**:

```
int sigaction(int signo, const struct sigaction *act, struct sigaction *oact);
```

signo — номер (имя) сигнала, для которого устанавливается диспозиция;

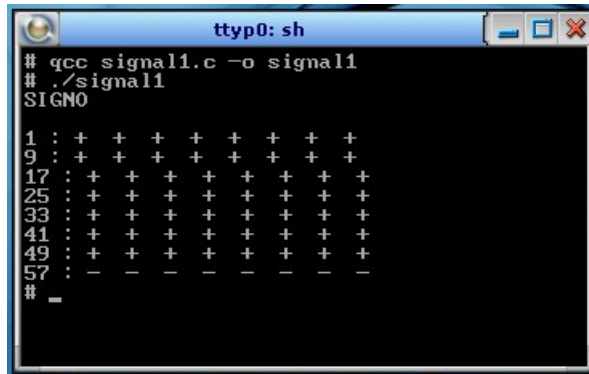
act — определение нового обработчика сигнала;

`oact` — структура (если указано не `NULL`), где будет сохранено описание ранее установленного обработчика (например, для последующего восстановления реакции).

Если `act` не `NULL`, тогда модифицируется указанный сигнал. Если `oact` не `NULL`, то предыдущее действие сохраняется в структуре, на которую он указывает. Использование комбинации `act` и `oact` позволяет запрашивать или устанавливать (либо и то и другое) действия сигналу.

Пример 2 (реализующий рекомендацию *POSIX*):

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <signal.h>
int main(int argc, char *argv[])
{
    printf ("SIGNO \n");
    int i = _SIGMIN;
    while(i <= _SIGMAX)
    {
        if (i % 8 == 1) printf ("\n%d :", i);
        int res = sigaction(i, NULL, NULL);
        if(res != 0 && errno == EINVAL)
        {
            printf (" - ");
        }
        else
        {
            printf (" + ");
        }
        i++;
    }
    printf ("\n");
    return EXIT_SUCCESS;
}
```

```
ttyp0: sh
# gcc signal1.c -o signal1
# ./signal1
SIGNO
1 : + + + + + + + +
9 : + + + + + + + +
17 : + + + + + + + +
25 : + + + + + + + +
33 : + + + + + + + +
41 : + + + + + + + +
49 : + + + + + + + +
57 : - - - - - - - -
#
```

Система считает все сигналы 1...56 реализуемыми, а последние 8 специфических сигналов [QNX](#), не допускают применения к ним `sigaction()`.

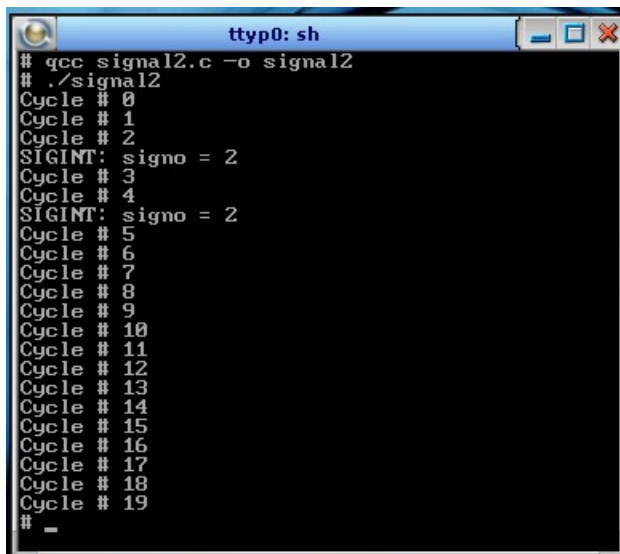
Пример 3 (запрет прерывания выполнения программы):

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <signal.h>
#include <unistd.h>

int main(void){
    extern void handler();
    struct sigaction act;
    sigset_t set;
    sigemptyset(&set); // формируем сигнальный набор,
    sigaddset(&set, SIGINT); // состоящий из двух сигналов:
    sigaddset(&set, SIGQUIT); // SIGINT и SIGQUIT.
    act.sa_flags = 0;
    act.sa_mask = set;
    act.sa_handler = &handler;
    sigaction(SIGINT, &act, NULL);
    sigaction(SIGQUIT, &act, NULL);
    int i = 0;
    while(i < 20){
        sleep(1);
        printf("Cycle # %d\n", i);
        i++;
    }
    return EXIT_SUCCESS;
}
```

Результатом нормального (без вмешательства оператора) выполнения приложения будет последовательность из 20 циклов секундных ожиданий, но если

в процессе этих ожиданий пользователь пытается прервать работу процесса по `[Ctrl+C]`, то он получит вывод, подобный следующему:



```
ttyp0: sh
# gcc signal2.c -o signal2
# ./signal2
Cycle # 0
Cycle # 1
Cycle # 2
SIGINT: signo = 2
Cycle # 3
Cycle # 4
SIGINT: signo = 2
Cycle # 5
Cycle # 6
Cycle # 7
Cycle # 8
Cycle # 9
Cycle # 10
Cycle # 11
Cycle # 12
Cycle # 13
Cycle # 14
Cycle # 15
Cycle # 16
Cycle # 17
Cycle # 18
Cycle # 19
# -
```

Функция `sigemptyset()` инициализирует пустой набор сигналов:

```
#include <signal.h>

int sigemptyset(sigset_t *set);
```

Инициализирует набор `set`, исключая из него все сигналы. Возвращает — 0, при удачном исполнении; -1, в случае ошибки.

Функция `sigaddset()` — добавляет в инициализированный набор `set` единственный сигнал `signo`:

```
#include <signal.h>

int sigaddset(sigset_t *set, int signo);
```

Присвоение сигнала набору `sigaddset()` возвращает — 0, при удачном исполнении; -1, в случае ошибки;